计算机组成与结构
—运算方法4

李瑞　　教授
蒋志平　副教授
计算机科学与技术学院

温故

- 简述先行进位加法器?
- 简述原码一位乘法原理?
- 简述原码两位乘法原理?
- 简述补码(booth)法乘法原理?
- 原码除法中，商符号怎么定？余数符号怎么定?
- 简述原码恢复余数除法?
- 简述加减交替除法，以及它的工作原理。

浮点运算—加减法

浮点运算—加减法

- 给定两个10进制科学计数法数字：
  - $X = 2.0230321 \times 10^8$
  - $Y = 2.0230321 \times 10^6$
- 你会如何做$X \pm Y$？
  - 对阶：$X \rightarrow Y$ or $Y \leftarrow X$
  - 加减运算

浮点运算—加减法

- 给定两个10进制科学计数法数字：
  - $X = 2.0230321 \times 10^{8}$
  - $Y = 2.0230321 \times 10^{6}$
- 你会如何做$X \pm Y$？
  - 对阶：$X \rightarrow Y$ or $Y \leftarrow X$
  - 加减运算

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶?
  - 运算?
  - then ?
  - done ?

浮点运算—加减法

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶？？
  - 运算？？？
  - then？？？
  - done？？？
- 如何对阶?
  - $X \rightarrow 0.000101010 \times 2^{1010}$
  - or $Y \rightarrow 110.110 \times 2^{111}$???
  - “小阶对大阶”，把小阶码数字对齐到大阶码
    - 因为：浮点表示法中，尾数只能是纯小数
  - 注意：阶码+1，尾数右移

浮点运算—加减法

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶
  - 运算？？？
  - then？？？
  - done？？？
- 如何运算（加减法）？
  - 万能解决方案：补码加减法！
- 搞定了？万事大吉？
  - $0.00011010101 \times 2^{1010}$，尾数非规格化
  - $01.01010101101 \times 2^{1010}$，尾数超出范围
- 怎么办？
  - 规格化！

浮点运算—加减法

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶
  - 运算
  - 规格化？？？
  - done？？？
- 如何规格化?
- 00.0xxxx或11.1xxxx类：
  - 尾数左移，直到尾数规格化；
  - 注意阶码是否出现下溢，若下溢，结果等效为0
- 01.xxxxx或10.xxxxx类：
  - 尾数右移，直到尾数规格化；
  - 注意阶码是否出现上溢，若上溢，结果等效为∞

这就结束了？！？？！

浮点运算—加减法

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶
  - 运算
  - 规格化
  - done？？？

- 如何规格化?
- 00.0xxxx或11.1xxxx类：
  - 尾数左移，直到尾数规格化；
  - 注意阶码是否出现下溢，若下溢，结果等效为0
- 01.xxxxx或10.xxxxx类：
  - 尾数右移，直到尾数规格化；
  - 注意阶码是否出现上溢，若上溢，结果等效为∞

这就结束了?

- $0.1010101101010... \times 2^{1000}$能存入给定位数浮点?

浮点运算—加减法

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \pm Y$？
  - 对阶
  - 运算
  - 规格化
  - 舍入处理

不考虑正负，可能有进位！

尾数超限，如何舍弃？

0.1010110001→ ???

- 方案一，粗暴截断：
  - 如0.1010100001→0.1010100
- 方案二，尾末置1：
  - 如0.1010100001→0.1010101
- 方案三，四舍五入(0舍1入)：
  - 如0.1010100001→0.1010100
  - 如0.1010101001→0.1010101
  - 如1.1010101001→1.1010110
  - 如0.1111111001→1.0000000

四舍五入导致再次溢出，应用截断法

浮点运算—加减法

- So, 浮点加减法4步操作：
  - 对阶（低阶对齐到高阶）
  - 运算（补码+/-法）
  - 规格化（左/右移）
  - 舍入处理（截断法/恒置1/四舍五入）

浮点运算—加减法

【例】阶码4位补码表示；尾数8位双符号位补码表示的浮点数X和Y，求和、差。

$X=0.110101\times2^{-010}$；$Y=-0.101010\times2^{-001}$

【解】

两数可表示为：

[X]浮＝1110; 00.110101

[Y]浮＝1111; 11.010110

- So, 浮点加减法4步操作：
  - 对阶（低阶对齐到高阶）
  - 运算（补码+/-法）
  - 规格化（左/右移）
  - 舍入处理（截断法/恒置1/四舍五入）

浮点运算—加减法

[X]浮＝1110; 00.110101

[Y]浮＝1111; 11.010110

1 对阶

需要显式地求阶差，根据阶差判断大小！

求阶差：

- **So, 浮点加减法4步操作：**
  - **对阶（低阶对齐到高阶）**
  - **运算（补码+/-法）**
  - **规格化（左/右移）**
  - **舍入处理（截断法/恒置1/四舍五入）**

[△E]补＝[EX]补＋[－EY]补＝1110＋0001＝1111

X的阶码比Y的阶码小。

X尾数右移一位，使两者阶码相同。这时的X为：

[X]’浮＝1111; 00.011010(1)

浮点运算—加减法

[X]浮＝1111; 00.0110101

[Y]浮＝1111; 11.010110

2 运算

[-Y]=1111;00.101010

- So, 浮点加减法4步操作：
  - 对阶（低阶对齐到高阶）
  - 运算（补码+/-法）
  - 规格化（左/右移）
  - 舍入处理（截断法/恒置1/四舍五入）

```
    0 0. 0 1 1 0 1 0 1          0 0. 0 1 1 0 1 0 1
+   1 1. 0 1 0 1 1 0      +     0 0. 1 0 1 0 1 0
-------------------------   -------------------------
    1 1. 1 1 0 0 0 0 1          0 1. 0 0 0 1 0 0 1
```

浮点运算—加减法

[X]浮＝1111; 00.0110101

[Y]浮＝1111; 11.010110

3 规格化

```
  0 0. 0 1 1 0 1 0 1          0 0. 0 1 1 0 1 0 1
+   1 1. 0 1 0 1 1 0        +   0 0. 1 0 1 0 1 0
  ---------------------       ---------------------
    1 1. 1 1 0 0 0 0 1          0 1. 0 0 0 1 0 0 1
```

左移，尾数左移2，

阶码减2

$[X+Y]_{浮}$＝1101;
11.000010

右移，尾数右移1，阶码加1

$[X+Y]_{浮}$＝0000;
00.10001001

舍去

- **So, 浮点加减法4步操作：**
  - **对阶（低阶对齐到高阶）**
  - **运算（补码+/-法）**
  - **规格化（左/右移）**
  - **舍入处理（截断法/恒置1/四舍五入）**

规格化左/右移目标：

正数目标:0.1xxx..

负数目标:1.0xxx..

浮点运算—加减法

【例】X和Y为浮点数，阶码4位，补码表示；尾数8位，双符号位补码表示，求X+Y和X-Y，舍入用四舍五入法。

X＝0110；00.101101；Y=0111；00.101100

- So, 浮点加减法4步操作：
  - 对阶（低阶对齐到高阶）
  - 运算（补码+/-法）
  - 规格化（左/右移）
  - 舍入处理（截断法/恒置1/四舍五入）

浮点运算—加减法

【例】X和Y为浮点数，阶码4位，补码表示；尾数8位，双符号位补码表示，求X+Y和X-Y，舍入用四舍五入法。

X＝0110；00.101101；Y=0111；00.101100

- So, 浮点加减法4步操作：
  - 对阶（低阶对齐到高阶）
  - 运算（补码+/-法）
  - 规格化（左/右移）
  - 舍入处理（截断法/恒置1/四舍五入）

解：$[-Y_{阶}]_{补}=1001$　$[-Y_{尾}]=11.010100$

$[X_{阶}]_{补}+[-Y_{阶}]_{补}=1111<0$

⇒将X对阶至$[Y_{阶}]=0111$

⇒$X^*=0111$；00.010110

X+Y：

00.010110

00.101100

01.000010

⇒$[X^*+Y]_{补}=01.000010$

右规，$[X^*+Y]=00.100001$ 0

四舍五入，$[X^*+Y]=00.100001$

X-Y：

00.010110

11.010100

11.101010

⇒$[X^*-Y]_{补}=11.101010$

左规：$[X^*-Y]_{补}=11.010100$

无舍入。

⇒$X-Y=11.010100$

浮点运算—乘除法

浮点运算—乘除法

- 给定两个10进制科学计数法数字：
  - $X = 2.0220322 \times 10^8$
  - $Y = 2.0220322 \times 10^6$
- 你会如何做$X \times Y$和$X \div Y$？
  - 底数x/÷底数
  - 阶数+/-阶数

浮点运算—乘除法

- 给定两个10进制科学计数法数字：
  - $X = 2.0200401 \times 10^{8}$
  - $Y = 2.0200401 \times 10^{6}$
- 你会如何做$X \times Y$和$X \div Y$？
  - 底数x/÷底数
  - 阶数+/-阶数

- 给定两个2进制浮点数：
  - $X = 0.101010 \times 2^{111}$
  - $Y = 0.110110 \times 2^{1010}$
- 你会如何做$X \times Y$和$X \div Y$？
  - 尾数x/÷尾数
  - 阶数+/-阶数
  - 规格化
  - 舍入

浮点运算—乘除法

【例】X和Y为浮点数，阶码4位，　　解：　[-|X尾|]补=11.010011

补码表示；尾数8位，双符号位补码

表示，求$X \times Y$，要求使用原码两位

乘法

X＝0100；00.101101；

Y=0011；00.101100

符号	D	Y	Y_1	Y_0	操作说明

- So, 浮点乘除法4步操作：
  - 尾数x/÷尾数
  - 阶数+/-阶数
  - 规格化
  - 舍入处理

浮点运算—乘除法

【例】X和Y为浮点数，阶码4位，补码表示；尾数8位，双符号位补码表示，求$X \times Y$，要求使用原码两位乘法

X＝0100；00.101101；

Y=0011；00.101100

解： [-|X尾|]补=11.010011

符号			D						Y				Y_1	Y_0	操作说明
0	0	0	0	0	0	0	0	0	1	0	1	1	0	0	D/Y初始化
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	尾数00，前无进位，+0，不进位
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	D/Y右移2位
1	1	1	0	1	0	0	1	1			1	0	1	1	尾数11，前无进位，-X，进位
1	1	1	0	1	0	0	1	1	0	0	1	0	1	1	
1	1	1	1	1	0	1	0	0	1	1	0	0	1	0	D/Y右移2位
1	1	1	0	1	0	0	1	1					1	0	尾数10，前有进位，-X，进位
0	0	0	0	0	0	1	1	1	1	1	0	0	1	0	
0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	D/Y右移2位
0	0	0	1	0	1	1	0	1							前有进位，+X
0	0	0	1	0	1	1	1	0	1	1	1	1	0	0	

- So, 浮点乘除法4步操作：
  - 尾数x/÷尾数
  - 阶数+/-阶数
  - 规格化
  - 舍入处理

|X尾| x |Y尾| = 0.101110111100

符号S = 00 XOR 00 = 00 =>

X尾 x Y尾= [00.101110111100]原

X阶+Y阶=0100+0011+1=1000，阶码溢出

浮点运算—乘除法

【例】X和Y为浮点数，阶码4位，　　解:

补码表示；尾数8位，双符号位补码

表示，求$X \div Y$，要求使用补码除法

X＝0100；00.101101；

Y=0011；11.101100

符号	余数（R）	商	操作说明

- So, 浮点乘除法4步操作：
  - 尾数x/÷尾数
  - 阶数+/-阶数
  - 规格化
  - 舍入处理

浮点运算—乘除法

【例】X和Y为浮点数，阶码4位，补码表示；尾数8位，双符号位补码表示，求$X \div Y$，要求使用补码除法

X＝0100；00.101101；

Y=0011；11.101100

非规格化！

- So, 浮点乘除法4步操作：
  - 尾数x/÷尾数
  - 阶数+/-阶数
  - 规格化
  - 舍入处理

解：X* = 0101; 00.0101101; Y*=0010; 11.011000;

[-Y*尾]补=00.101000;

被除数要小于除数！

符号	余数（R）							商	操作说明
0 0	0	1	0	1	1	0	1		R初始化
1 1	0	1	1	0	0	0			R与D异号，+D
1 1	1	0	1	1	1	0	1	1	R与D同号，商1
1 1	0	1	1	1	0	1	0		R左移1位
0 0	1	0	1	0	0	0			R与D同号，-D
0 0	0	0	0	1	0	1	0	0	R与D异号，商0
0 0	0	0	1	0	1	0	0		R左移1位
1 1	0	1	1	0	0	0			R与D异号，+D
1 1	1	0	0	0	1	0	0	1	R与D同号，商1
1 1	0	0	0	1	0	0	0		R左移1位
0 0	1	0	1	0	0	0			R与D同号，-D
1 1	1	0	1	1	0	0	0	1	R与D同号，商1
1 1	0	1	1	0	0	0	0		R左移1位
0 0	1	0	1	0	0	0			R与D同号，-D
0 0	0	0	0	0	0	0	0	1	除尽，末位置1

$X \div Y$ = 0011；11.011100

运算的硬件实现
—从运算器、ALU到CPU

运算的硬件实现
—从运算器、ALU到CPU

- 运算器
  - 加法器（运算方法1中已讲）
  - 乘法器
  - 除法器
- ALU

运算硬件实现——阵列乘法器

设$X=X_3X_2X_1X_0$，$Y=Y_3Y_2Y_1Y_0$，计算$X\cdot Y=?$

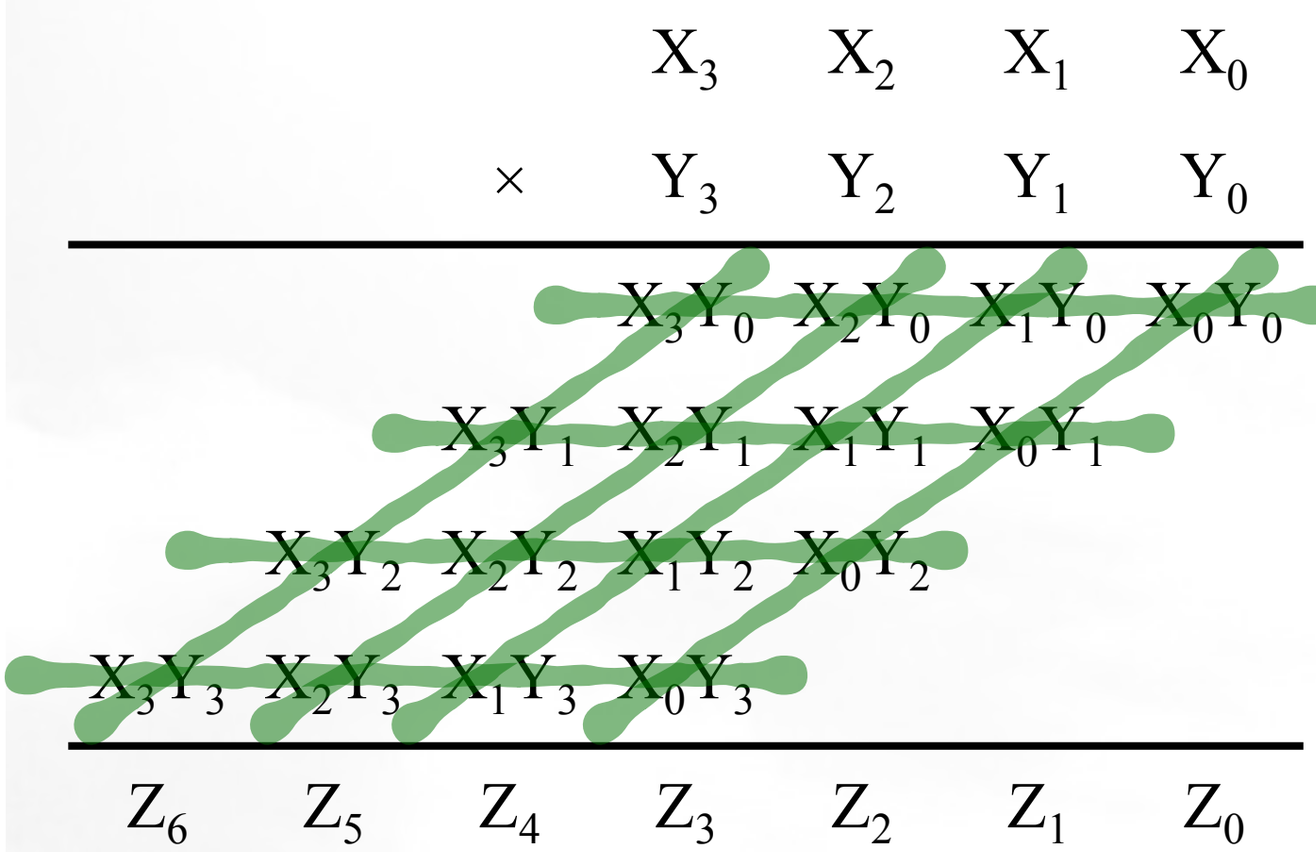
$$
\begin{array}{cccccccc}
 & & X_3 & X_2 & X_1 & X_0 & & \\
\times & & Y_3 & Y_2 & Y_1 & Y_0 & & \\
\hline
 & & X_3Y_0 & X_2Y_0 & X_1Y_0 & X_0Y_0 & & \\
 & X_3Y_1 & X_2Y_1 & X_1Y_1 & X_0Y_1 & & & \\
 & X_3Y_2 & X_2Y_2 & X_1Y_2 & X_0Y_2 & & & \\
X_3Y_3 & X_2Y_3 & X_1Y_3 & X_0Y_3 & & & & \\
\hline
Z_6 & Z_5 & Z_4 & Z_3 & Z_2 & Z_1 & Z_0 &
\end{array}
$$

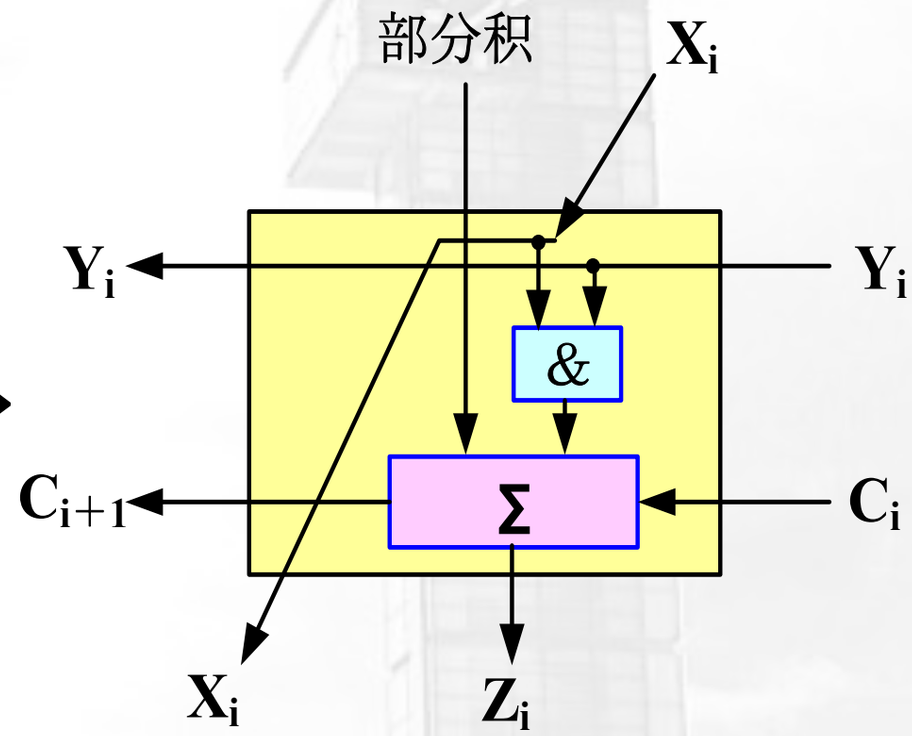
与运算

求和

运算硬件实现——阵列乘法器

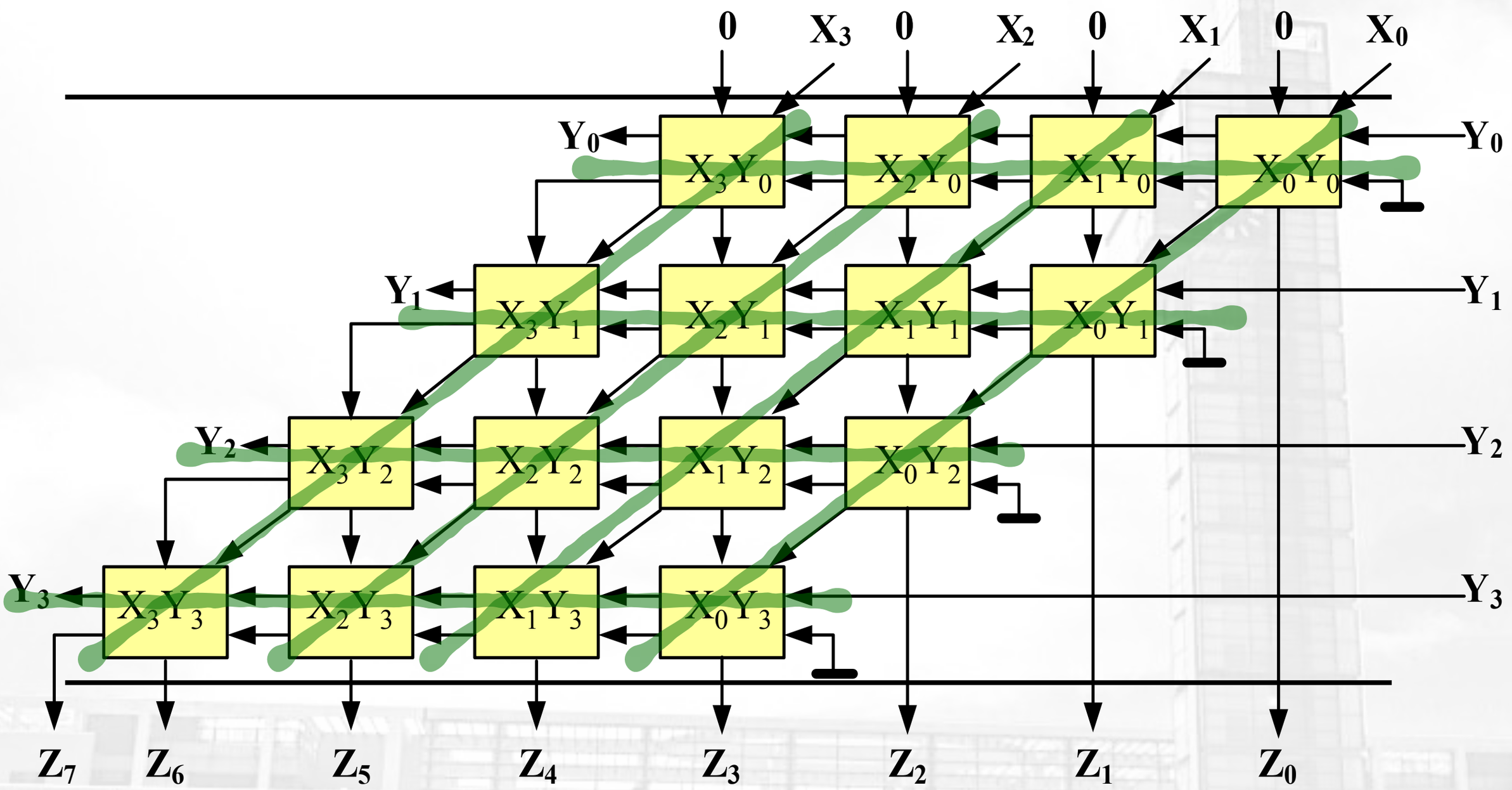
设$X=X_3X_2X_1X_0$，$Y=Y_3Y_2Y_1Y_0$，计算$X\cdot Y=?$





基本乘加单元框图

运算硬件实现——阵列乘法器



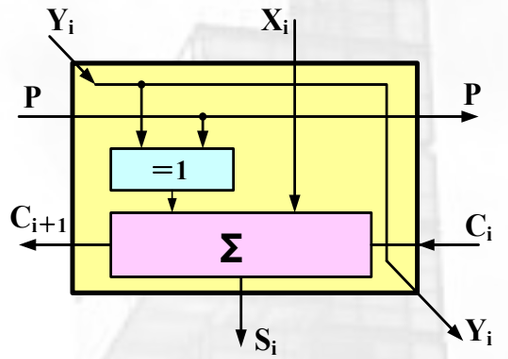
运算硬件实现——阵列乘法器

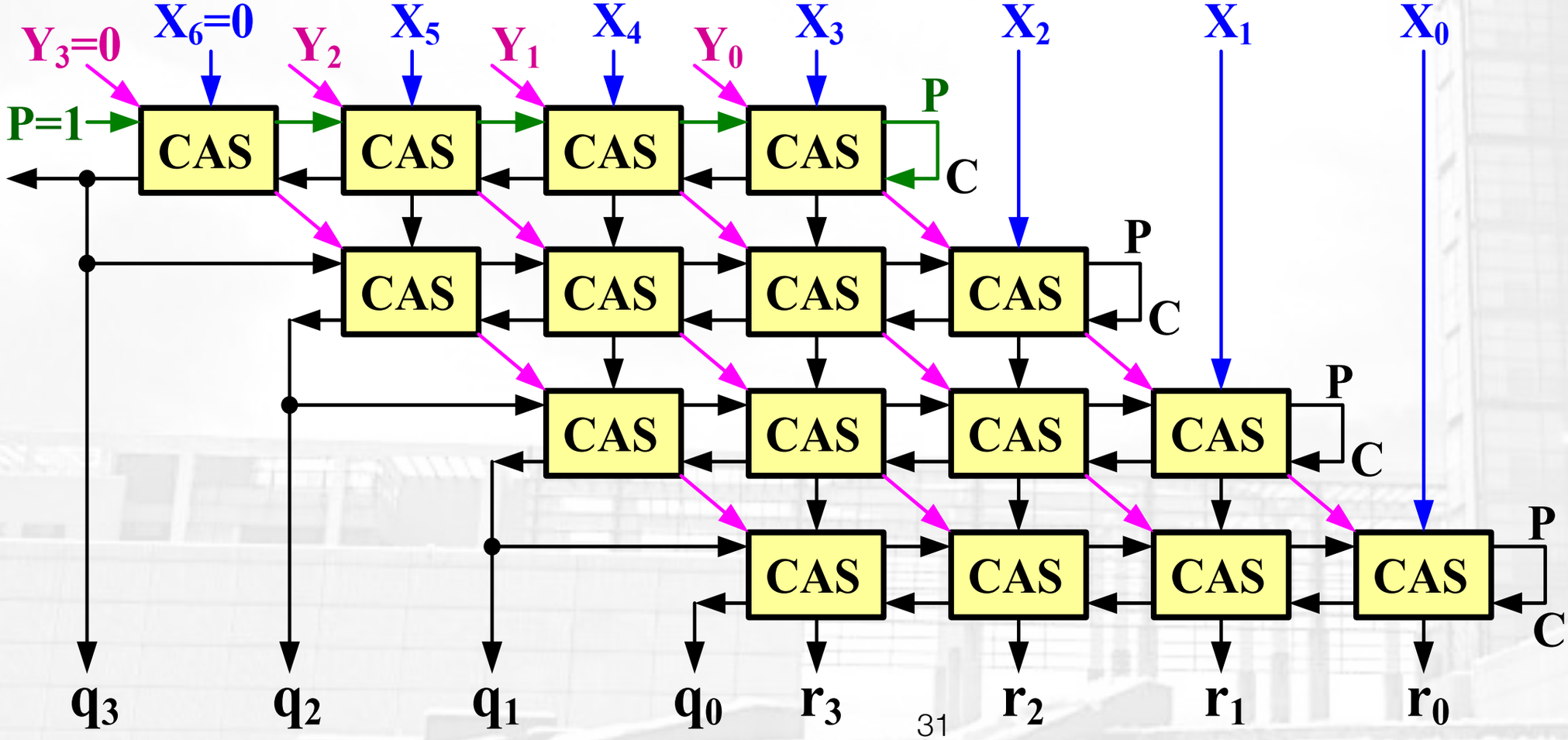
- 计算性能的神器：Fused Multiply-Add (FMA)操作！
  - FMA可以在一个时钟内算出一个乘法！
- SSE1/2/3/4，AVX，FMA1/2/3/4等等高级指令级都是基于大规模、并发、流水线化的FMA阵列。
-

运算硬件实现——阵列除法器

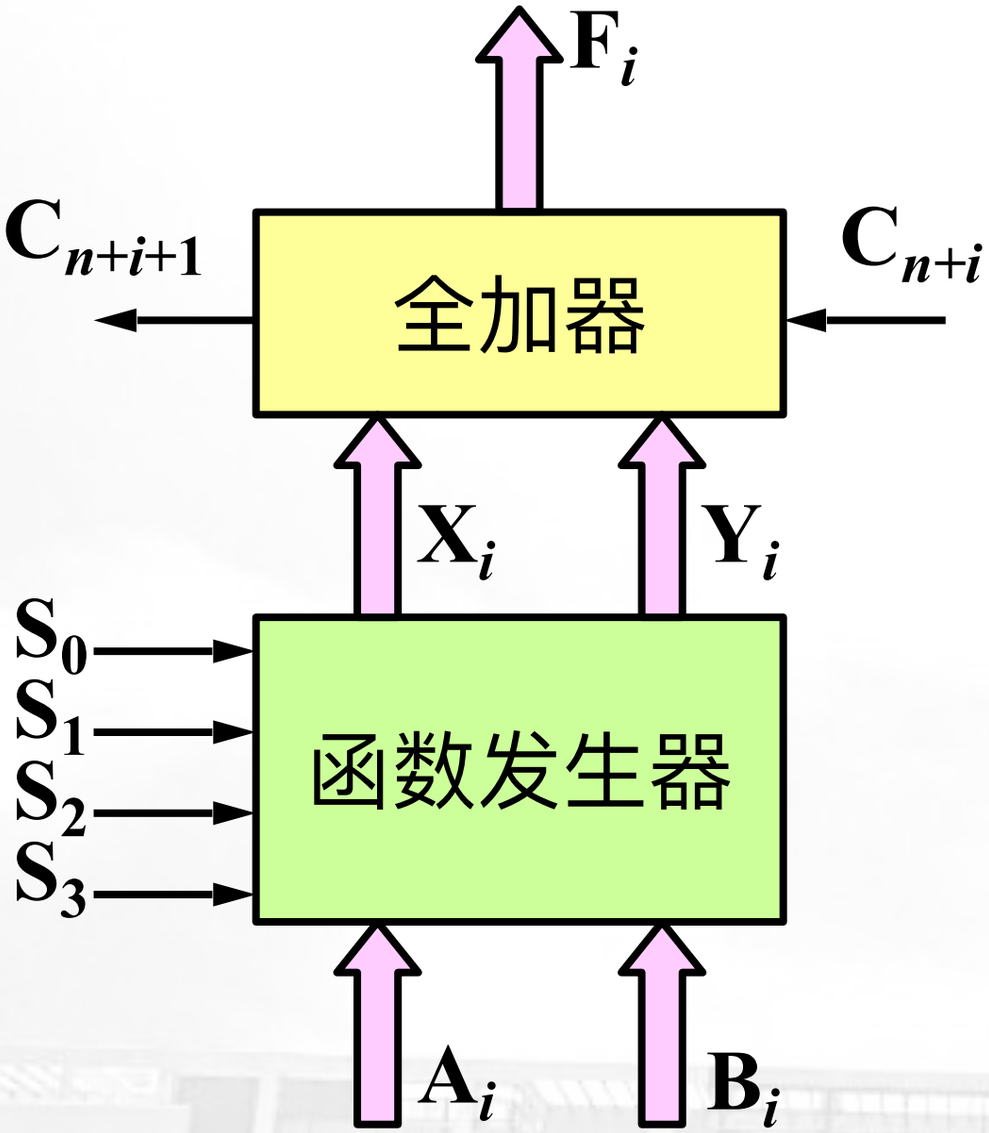
设$X=X_3X_2X_1X_0$，$Y=Y_3Y_2Y_1Y_0$，计算X/Y＝？

可控加/减单元CAS





运算的硬件实现——ALU



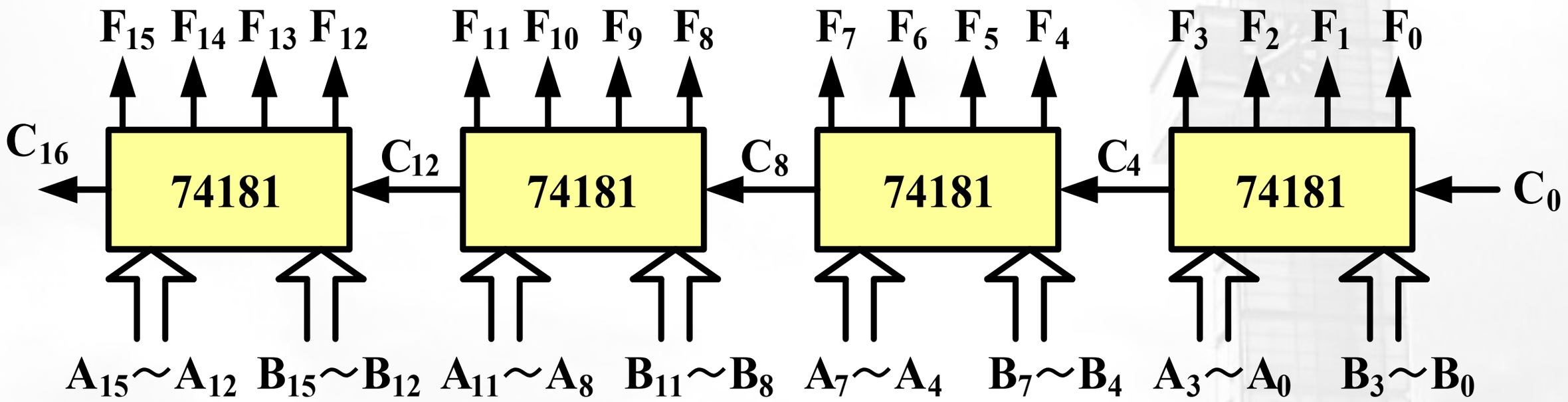
Selection				Active High Data		
				M = H	M = L; Arithmetic Operations	
S3	S2	S1	S0	Logic Functions	C_n = H (no carry)	C_n = L (with carry)
L	L	L	L	F = $\overline{A}$	F = A	F = A Plus 1
L	L	L	H	F = $\overline{A + B}$	F = A + B	F = (A + B) Plus 1
L	L	H	L	F = $\overline{A}$B	F = A + $\overline{B}$	F = (A + $\overline{B}$) Plus 1
L	L	H	H	F = 0	F = Minus 1 (2's Compl)	F = Zero
L	H	L	L	F = $\overline{AB}$	F = A Plus A$\overline{B}$	F = A Plus A$\overline{B}$ Plus 1
L	H	L	H	F = $\overline{B}$	F = (A + B) Plus A$\overline{B}$	F = (A + B) Plus A$\overline{B}$ Plus 1
L	H	H	L	F = A ⊕ B	F = A Minus B Minus 1	F = A Minus B
L	H	H	H	F = A$\overline{B}$	F = A$\overline{B}$ Minus 1	F = A$\overline{B}$
H	L	L	L	F = $\overline{A}$ + B	F = A Plus AB	F = A Plus AB Plus 1
H	L	L	H	F = $\overline{A \oplus B}$	F = A Plus B	F = A Plus B Plus 1
H	L	H	L	F = B	F = (A + $\overline{B}$) Plus AB	F = (A + $\overline{B}$) Plus AB Plus 1
H	L	H	H	F = AB	F = AB Minus 1	F = AB
H	H	L	L	F = 1	F = A Plus A*	F = A Plus A Plus 1
H	H	L	H	F = A + $\overline{B}$	F = (A + B) Plus A	F = (A + B) Plus A Plus 1
H	H	H	L	F = A + B	F = (A + $\overline{B}$) Plus A	F = (A + $\overline{B}$) Plus A Plus 1
H	H	H	H	F = A	F = A Minus 1	F = A

*Each bit is shifted to the next more significant position.

运算的硬件实现——ALU

组内先行进位
组间串行进位

级联工作：



由74181构成组间串行进位的16位ALU

运算的硬件实现——ALU

先行进位部件(**CLA**)：**74LS182**

假设**4**片**74LS181**的先行进位输出依次为：

$$P_0,\ G_0,\ P_1,\ G_1,\ P_2,\ G_2,\ P_3,\ G_3$$

$$C_{n+x} = G_0 + P_0 C_n$$

$$C_{n+y} = G_1 + P_1 C_{n+x} = G_1 + G_0 P_1 + P_0 P_1 C_n$$

$$C_{n+z} = G_2 + P_2 C_{n+y} = G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_n$$

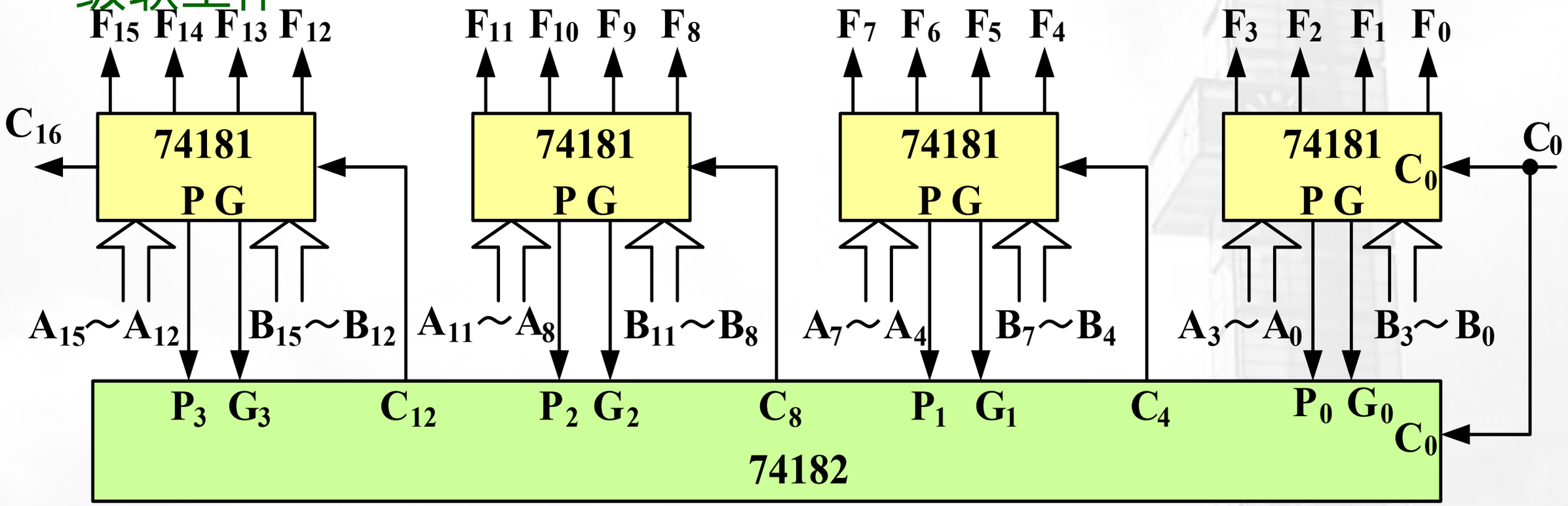
$$C_{n+4} = G_3 + P_3 C_{n+z} = \underbrace{G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3}_{G^*} + \underbrace{P_0 P_1 P_2 P_3}_{P^*} C_n$$

$P^* = P_0 P_1 P_2 P_3$ ……成组进位传送输出

$G^* = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$ ……成组进位发生输出

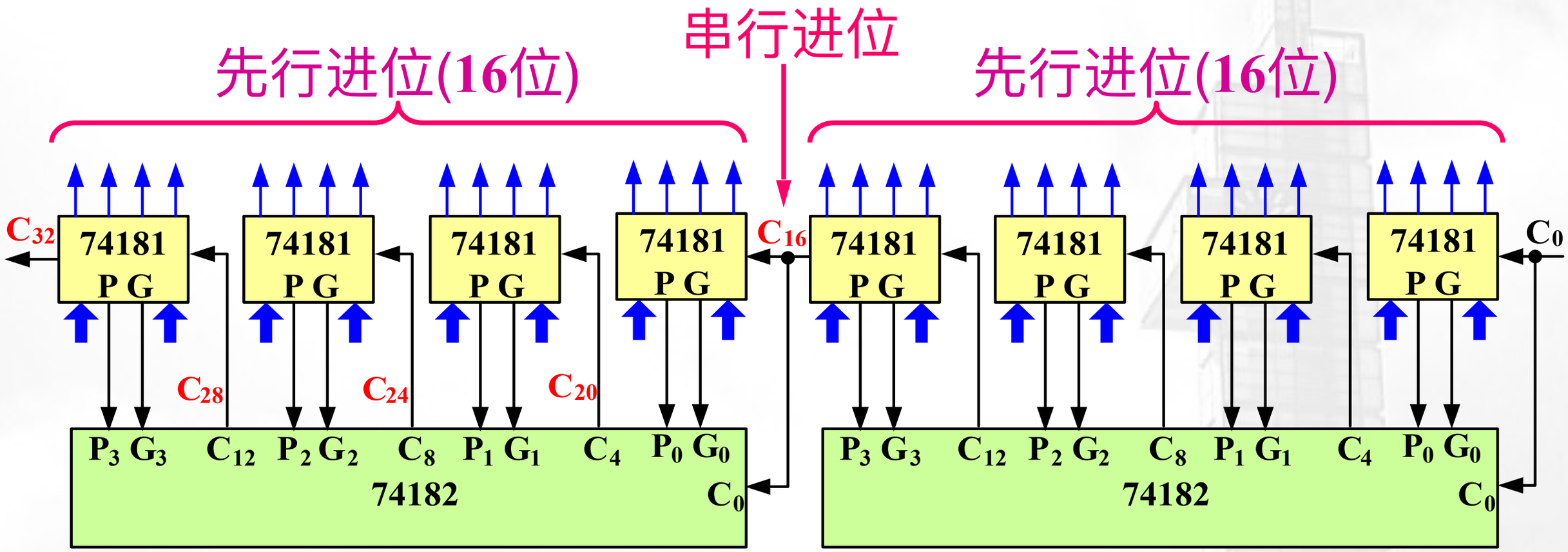
运算的硬件实现——ALU

级联工作：



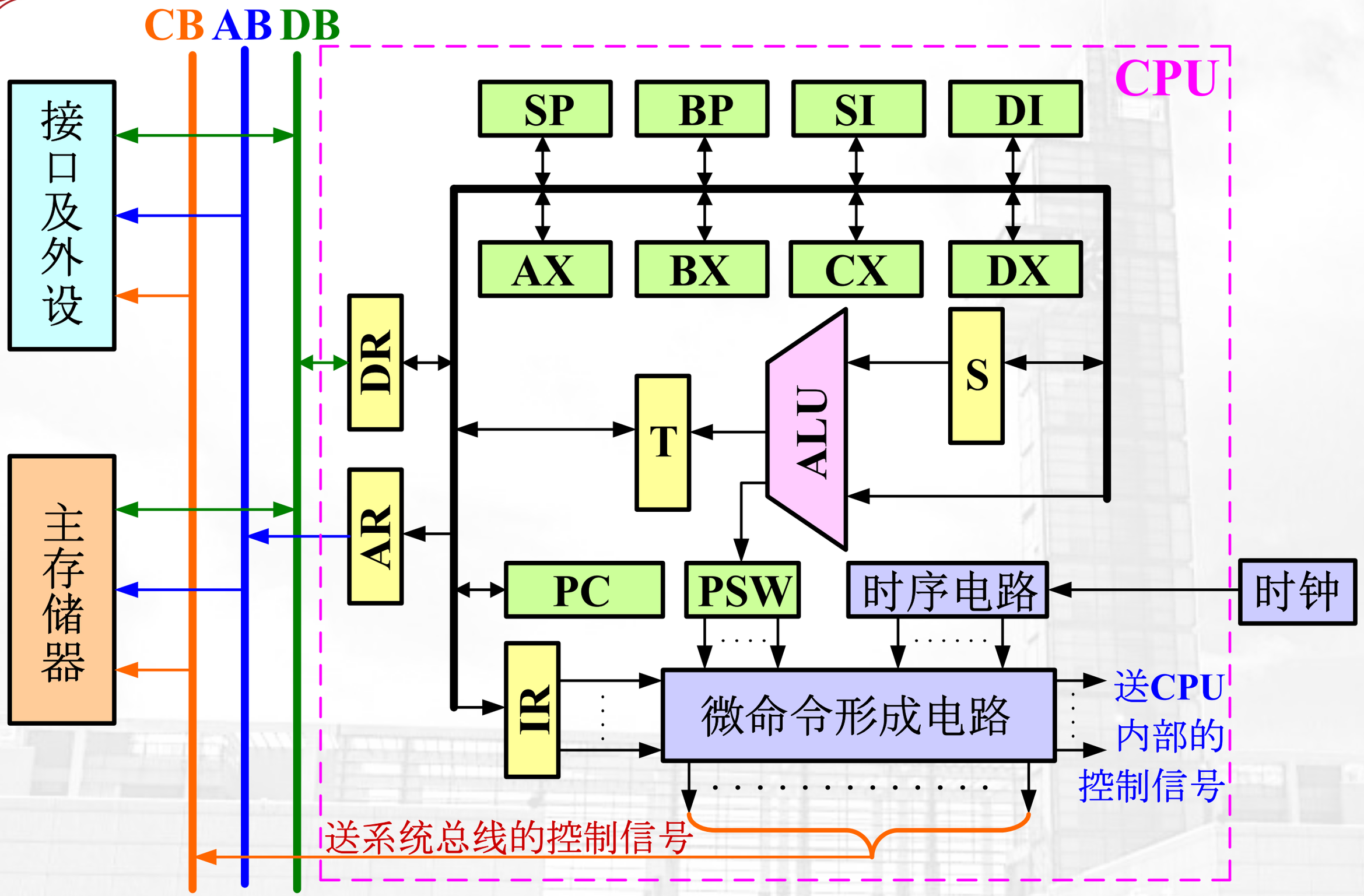
由74181和74182构成组内组间均并行进位的16位ALU

运算的硬件实现——ALU



用两个16位全先行进位逻辑级联组成的32位ALU

运算的硬件实现——CPU


CB AB DB
CPU
接口及外设
主存储器
SP
BP
SI
DI
AX
BX
CX
DX
DR
AR
T
ALU
S
PC
PSW
时序电路
时钟
IR
微命令形成电路
送CPU内部的控制信号
送系统总线的控制信号

Ariane 5 Disaster

- Ariana 5号火箭，研发10余年，耗资$7B
- 96年6月4日，发射，摔了
  - 无人，毁了$500M货物



Ariana 5 Disaster

- 软件bug导致!
- 具体原因:
  - 在惯性导航系统（INS）中，某个关于水平速度的64bit浮点数(double)在代码中被转换为16bit有符号整数(int16_t/short)；
- 在故障时刻，这个double数值已经超过32767，转int16时溢出~~~
- “Not Enough Space to Reach Space”



乘除法巩固练习

乘除法巩固练习（随堂小测试）

- 用原码两位乘法 和 Booth法计算：-0.11001101 x -0.1101

- 用原码不恢复法 和 补码除法计算：-0.10100110 / -0.1101

乘除法巩固练习（随堂小测试）

- 答案：
  - 乘法：0.101001101001
  - 除法：